Веб-программирование: Основы React’а

Яндекс Образование

Содержание

01 Что такое React

02 Архитектура

03 JSX

04 Жизненный цикл

05 Работа с событиями

06 Q&A

Что такое React

React

React — это декларативная, компонентная JavaScript-библиотека для создания интерактивных пользовательских интерфейсов, использующая виртуальный DOM и однонаправленный поток данных для эффективного управления отображением и состоянием приложения.

- Библиотека, а не фреймворк
  - Не навязывает архитектуру проекта
  - Отвечает только за отрисовку интерфейса
- Разрабатывается и поддерживается Meta Platforms и сообществом разработчиков

Краткая история

- Создан в 2011 году инженером Facebook Джорданом Уолке (Jordan Walke) для решения внутренних задач компании (первоначальная цель - улучшить производительность и упростить поддержку кода новостной ленты Facebook)
- Май 2013 года — официальный релиз с открытым исходным кодом на конференции JSConfUS
- 2015 год — анонс ReactNative для разработки нативных мобильных приложений (iOS/Android)
- 2017год — презентация ReactFiber (движок обновления интерфейса)
- 2018год — введение хуков, сдвиг архитектуры от классовых к функциональным компонентам

Философия

- Компонентный подход
- Виртуальный DOM
- Однонаправленный поток данных
- Декларативность
- Узкая зона ответственности (View в MVC)

Типичный стек

- Ядро: React
- Управление состоянием: Redux, MobX, Zustand
- Маршрутизация: React Router
- Стилизация: CSS-модули, Sass, styled-components, Tailwind
- Сборка: Webpack, Vite, RollUp
- Мобильная разработка: ReactNative

Преимущества

- Гибкость: подходит для маленьких виджетов и масштабных приложений
- Экосистема: огромное количество готовых решений и инструментов
- Поддержка сообщества: активное развитие, регулярные обновления, обширная документация
- Низкий порог входа: сравнительно простой

Недостатки

- Нет единой структуры
- Нет встроенных решений для роутинга, http-запросов, стилей и т.д.
- JSX - это не HTML, это запутывает
- Из коробки SEO не очень хороший
- Слишком быстрая эволюция экосистемы
- Слишком большая экосистема
- Плохая производительность при неоптимальном коде
- Запутанное управление состоянием

Официальный сайт

https://react.dev

Архитектура

Яндекс Образование

Компоненты

Компонент в React — это независимый, повторно используемый фрагмент интерфейса, который описывает, как должна выглядеть и вести себя определённая часть веб-страницы.

- Самодостаточность
- Переиспользуемость
- Вложенность
- Принимает входные данные
- Возвращает UI-элемент

Пример

```jsx
import React from 'react';

function Welcome(props) {
  return (
    <div className="welcome-card">
      <h2>Привет, {props.name}!</h2>
      <p>Добро пожаловать в наше приложение.</p>
    </div>
  );
}
```

Классовые компоненты

Классовый компонент в React — это компонент, реализованный как класс JavaScript, наследующий от React.Component.

- Устарел
- Используется только в ErrorBoundary
- Иногда даёт более точечный контроль над жизненным циклом

Пример

```
import React from 'react';

class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  increment() {
    this.setState({ count: this.state.count + 1 });
  };

  componentDidMount() {
    console.log('Компонент добавлен в DOM');
  }

  render() {
    return (
      <div>
        <p>Счётчик: {this.state.count}</p>
        <button onClick={this.increment}>+1</button>
      </div>
    );
  }
}

export default Counter;
```

Функциональные компоненты

Функциональные компоненты — по сути, это обычные JavaScript-функции, которые:

- принимают входные данные (props)
- возвращают описание интерфейса (JSX)

- Простые в сравнении с классами
- Нет собственного состояния
- Чистые (следующий слайд)
- Производительные (меньше накладных расходов по сравнению с классами)

Чистые функции

Чистая функция (pure function) — это функция, которая при вызове:

- всегда возвращает один и тот же результат для одних и тех же входных аргументов
- не оказывает побочных эффектов (side effects) на состояние программы или внешнюю среду

Пример

- Без side effects

```
function double(x) {
  return x * 2;
}
```

- С side effects (не чистая)

```
let counter = 0;
function increment() {
  counter++; // изменяет внешнюю переменную
  return counter;
}
```

Стрелочные функции

Стрелочные функции (arrow functions) — это современный синтаксис для объявления функций в JavaScript (введён в стандарте ES6). Они делают код компактнее и решают ряд проблем, связанных с контекстом this.

```
(параметры) => { тело функции }

const add = (a, b) => a + b
```

Особенности

- Краткий синтаксис: нет ключевого слова function
- Неявный возврат: можно опускать return, если тело функции - одно выражение
- Нет собственного this
- Нет объекта arguments
- Нельзя использовать как конструктор
- Захватывает переменные из внешней области видимости

Преимущества в сравнении с классами

- Меньше кода — нет constructor, super, this
- Проще читать и понимать — чистая функция с входными данными и результатом
- Лучшая поддержка хуков — изначально спроектированы для работы с useState, useEffect и др
- Легче рефакторить — можно легко разбивать на мелкие компоненты

Правила

- Имя компонента — с заглавной буквы (MyComponent, а не mycomponent).
- Возвращает один корневой элемент — всё содержимое должно быть обернуто в один тег (например,).
- Props — только для чтения — нельзя изменять props внутри компонента.
- Хуки — только в функциональных компонентах — не использовать в обычных функциях или классовых компонентах.

Пример

```
const MyComponent = (props) => (
  <div>
    <h1>Привет, {props.name}!</h1>
    <p>Это функциональный компонент.</p>
  </div>
);
```

Передача данных в компонент: props

Props (сокращение от properties — свойства) — основной механизм передачи данных от родительского компонента к дочернему в React. Это неизменяемые (immutable) данные, которые компонент получает извне и использует для отрисовки или логики.

- Только для чтения
- Передача разных типов данных
- Деструктуризация
- Можно задавать значения по умолчанию

Сравнение props

Сравнение props в React — ключевой механизм, обеспечивающий оптимальный перерендеринг компонентов. React сравнивает пропсы при каждом обновлении, чтобы решить: нужно ли перерисовывать компонент и его потомков.

- Сравнение пропсов работает через ===
- Типы не приводятся
- Объекты, массивы и функции сравниваются **по ссылке**

Собственное состояние компонента: state

State — это внутренние данные компонента, которые могут меняться со временем и вызывают перерисовку интерфейса при обновлении. В отличие от props (которые передаются извне и неизменяемы), state управляется внутри самого компонента.

Правила

- Обновляйте state только через useState
- Обновления могут быть асинхронными
- React может группировать несколько вызовов setState в один рендер
- В пределах одного рендера значение state не меняется, даже если вызван useState

Композиция компонентов

Композиция — фундаментальный принцип React, позволяющий строить сложные интерфейсы из небольших, переиспользуемых компонентов-«кирпичиков». Вместо наследования React предпочитает композицию: компоненты собирают, а не расширяют.

Принципы

- Каждый компонент — одна ответственность
- Компоненты должны быть небольшими и понятными
- Используйте props для связи
- Используйте children для гибкости

Пример

```
function Container({ children }) {
  return <div className="container">{children}</div>;
}

// Использование
<Container>
  <h1>Заголовок</h1>
  <p>Содержимое</p>
</Container>
```

Хуки

Хуки (hooks) — это функции в React, позволяющие «подключать» к функциональным компонентам возможности, ранее доступные только в классовых компонентах: управление состоянием, побочные эффекты, контекст и др. Введены в версии React 16.8.

useState

useState — встроенный хук React, позволяющий функциональным компонентам хранить и обновлять внутреннее состояние

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Счётчик: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

useEffect

useEffect — встроенный хук React, позволяющий выполнять побочные эффекты (side effects) в функциональных компонентах. Побочные эффекты — это операции, которые выходят за рамки чистого рендеринга: запросы к API, подписки, манипуляции с DOM, таймеры и т. п.

```
useEffect(() => {
  const timer = setInterval(() => {
    setCount(count + 1);
  }, 1000);

  // Очищаем интервал при размонтировании
  return () => clearInterval(timer);
}, [count]);
```

useCallback

useCallback — хук React для мемоизации функций (сохранения ссылки на функцию между рендерами). Он помогает оптимизировать производительность, предотвращая ненужные пересоздания функций и перерендеры дочерних компонентов.

```
const handleSubmit = useCallback(() => {
  saveData(currentValue);
}, [currentValue]);
```

useMemo

useMemo — хук React для мемоизации (кеширования) вычисляемых значений. Он позволяет избежать повторных ресурсоёмких вычислений при рендере, сохраняя результат до изменения зависимостей.

```
function UserList({ users }) {
  const activeUsers = useMemo(
    () => users.filter(user => user.isActive),
    [users]
  );

  return /* какой-то код, использующий activeUsers */;
}
```

useRef

useRef — хук React для создания изменяемого объекта-контейнера, который сохраняется между рендерами компонента

- Данные хранятся в .current
- Изменение .current не вызывает ререндер
- Объект ref не изменяется между рендерами
- Можно изменять напрямую

Пример

```
function InputWithFocus() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus(); // Устанавливаем фокус при монтировании
    // inputRef.current = document.getElementById('input');
  }, []);

  return (
     <input
        ref={inputRef}
        placeholder="Введите текст"
        id=“input”
     />;
  );
}
```

Кастомные хуки

Кастомные хуки (custom hooks) — это пользовательские функции в React, которые позволяют выносить и повторно использовать логику с состоянием и побочными эффектами между компонентами

- Имя с префиксом use
- Могут вызывать другие хуки
- Нет JSX
- Можно использовать в разных компонентах

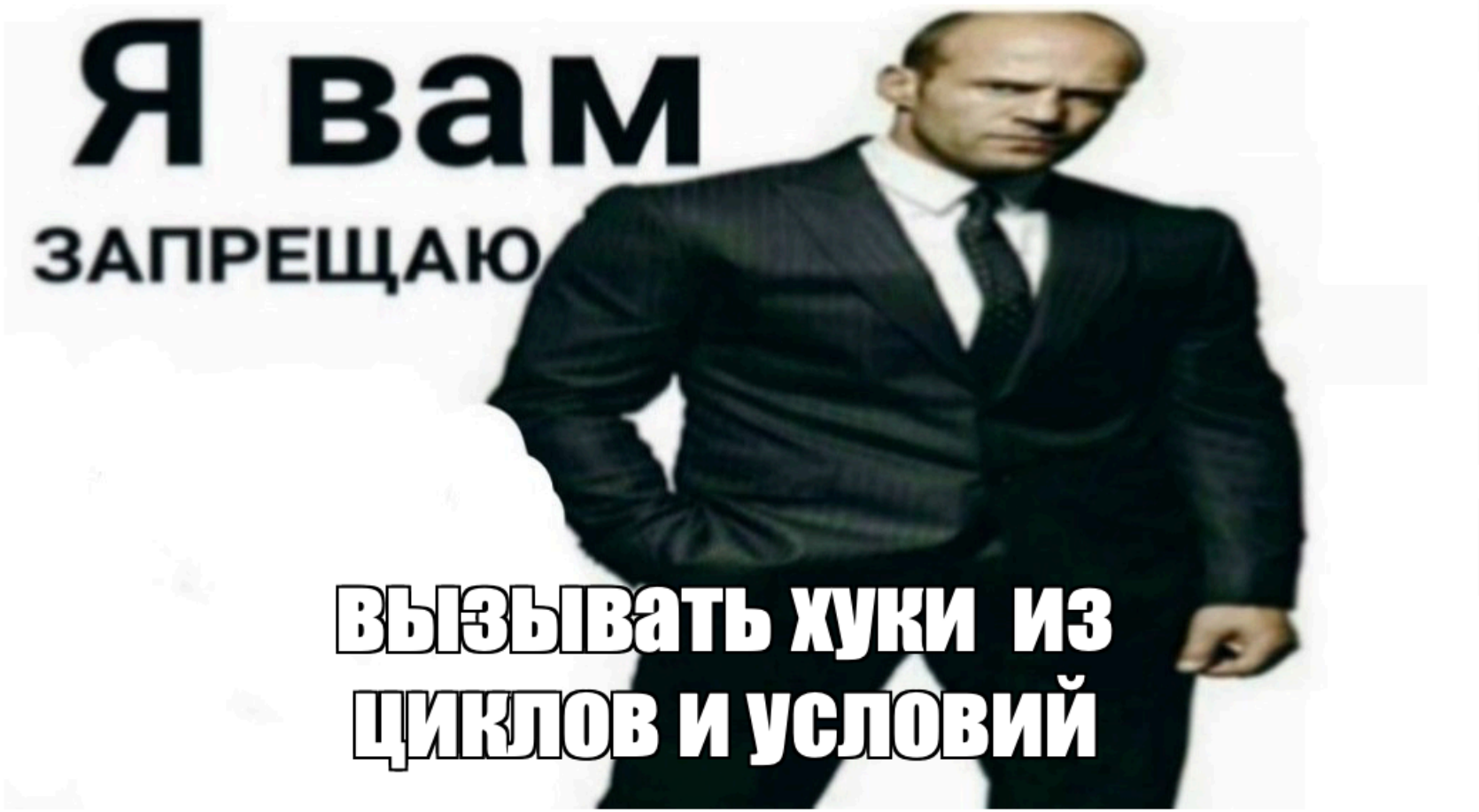
Пример

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    fetch(url)
      .then((res) => res.json())
      .then((json) => setData(json))
      .catch((err) => setError(err));
  }, [url]);

  return { data, error };
}
```

Правила

- Вызывайте хуки только на верхнем уровне: не помещайте их в условия, циклы или вложенные функции.
- Соблюдайте порядок вызовов: хуки должны вызываться в одной и той же последовательности при каждом рендере.
- Используйте массивы зависимостей ([] в useEffect, useMemo, useCallback)
  - Укажите все значения, от которых зависит хук.
  - Пустой массив [] означает выполнение только при монтировании.
- Не вызывайте хуки внутри обычных функций: только в функциональных компонентах или других хуках.
- Избегайте лишних зависимостей: включайте в массив зависимостей только те значения, которые реально используются.

Правила



Плюсы

- Упрощают код — меньше вложенности и обёрток по сравнению с HOC и Render Props.
- Улучшают переиспользование логики — кастомные хуки легко тестировать и применять в разных компонентах.
- Делают компоненты более читаемыми — логика группируется по назначению, а не по жизненному циклу.

Минусы

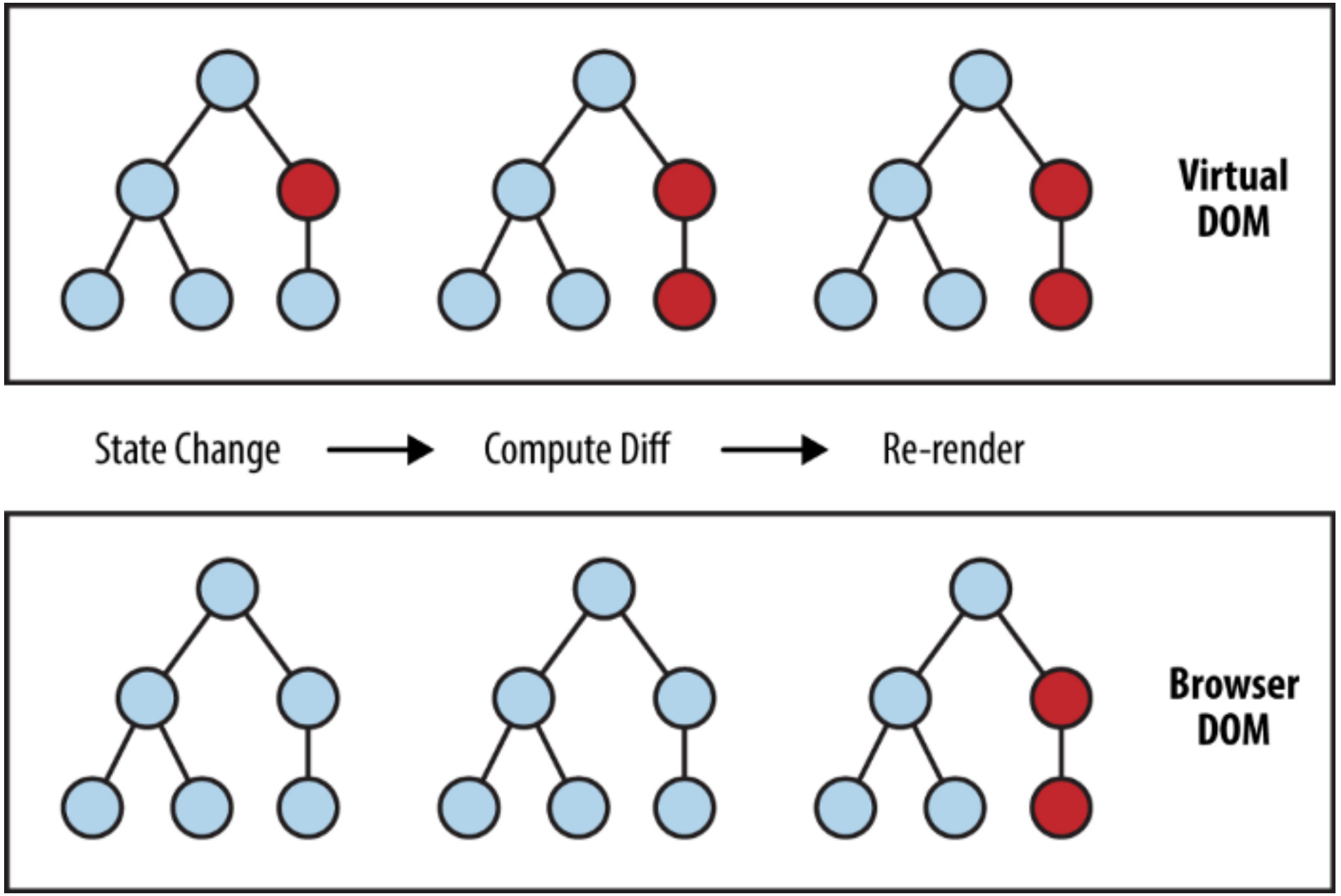
- Не заменяют классы полностью — некоторые паттерны (например, Error Boundaries) требуют классовых компонентов.
- Состояние не делится между компонентами — каждый компонент получает свою копию состояния из хука.
- Требуют понимания зависимостей — неправильные массивы зависимостей приводят к багам.
- Не работают в классовых компонентах — только в функциональных.

Виртуальный DOM

Виртуальный DOM — это облегчённая копия реального DOM (Document Object Model), хранящаяся в памяти. Это JavaScript-объект, который отражает структуру HTML-документа, но не взаимодействует напрямую с браузером.

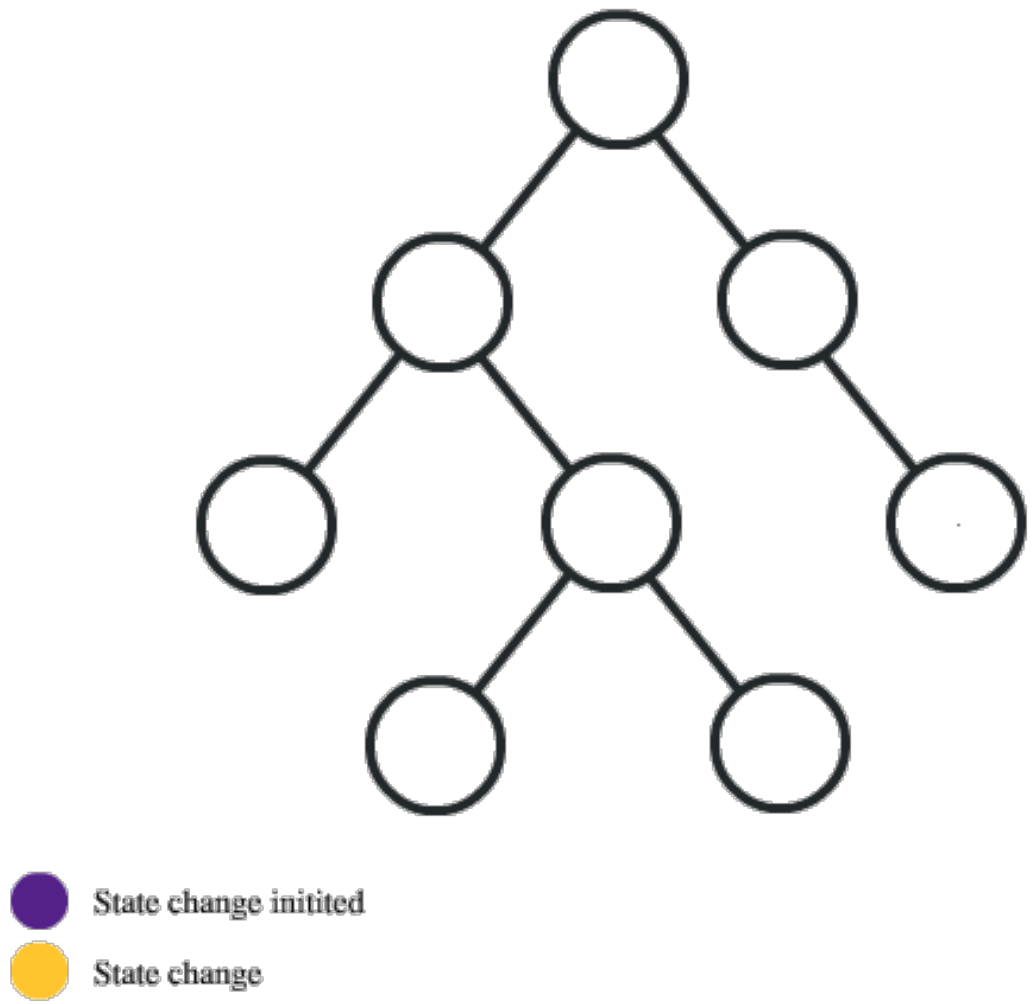
Работа по шагам

1. Начальный рендеринг
   При запуске создаём структуру всего DOM-дерева в памяти
2. Меняются props или state
   Создаём новую структуру DOM-дерева в памяти
3. Сравниваем старое и новое (diffing)
   Помечаем, что добавилось, изменилось или удалилось
4. Reconciliation (согласование) и патчинг
   Меняем только то, что нужно, не трогая остальное реальное DOM-дерево


Virtual
DOM
State Change
Compute Diff
Re-render
Browser
DOM

Однонаправленный поток данных

Однонаправленный поток данных — это архитектурный принцип, при котором данные движутся в строго определённом направлении: от родительских компонентов к дочерним. Любые изменения состояния инициируются в одном месте и распространяются предсказуемо вниз по дереву компонентов.


State change initited
State change

Пример

```
function Parent() {
  const [count, setCount] = useState(0);
  const increment = () => setCount(count + 1);
  return <Child onIncrement={increment} count={count} />;
}

function Child({ onIncrement, count }) {
  return (
    <div>
      <p>Счёт: {count}</p>
      <button onClick={onIncrement}>+1</button>
    </div>
  );
}
```

Context API

Context API — встроенный в React механизм для передачи данных между компонентами без проброса пропсов через промежуточные уровни

Когда применять:

- Глобальные настройки (тема, язык, регион).
- Данные аутентификации (текущий пользователь, статус входа).
- Общие сервисы (экземпляры API-клиентов, логгеры).
- Состояние приложения, нужное во многих компонентах (корзина, уведомления).
- Глубоко вложенные компоненты, которым нужны одни и те же данные.

Шаг 1: создание контекста

```
import { createContext } from 'react';

export const LevelContext = createContext(‘light');
```

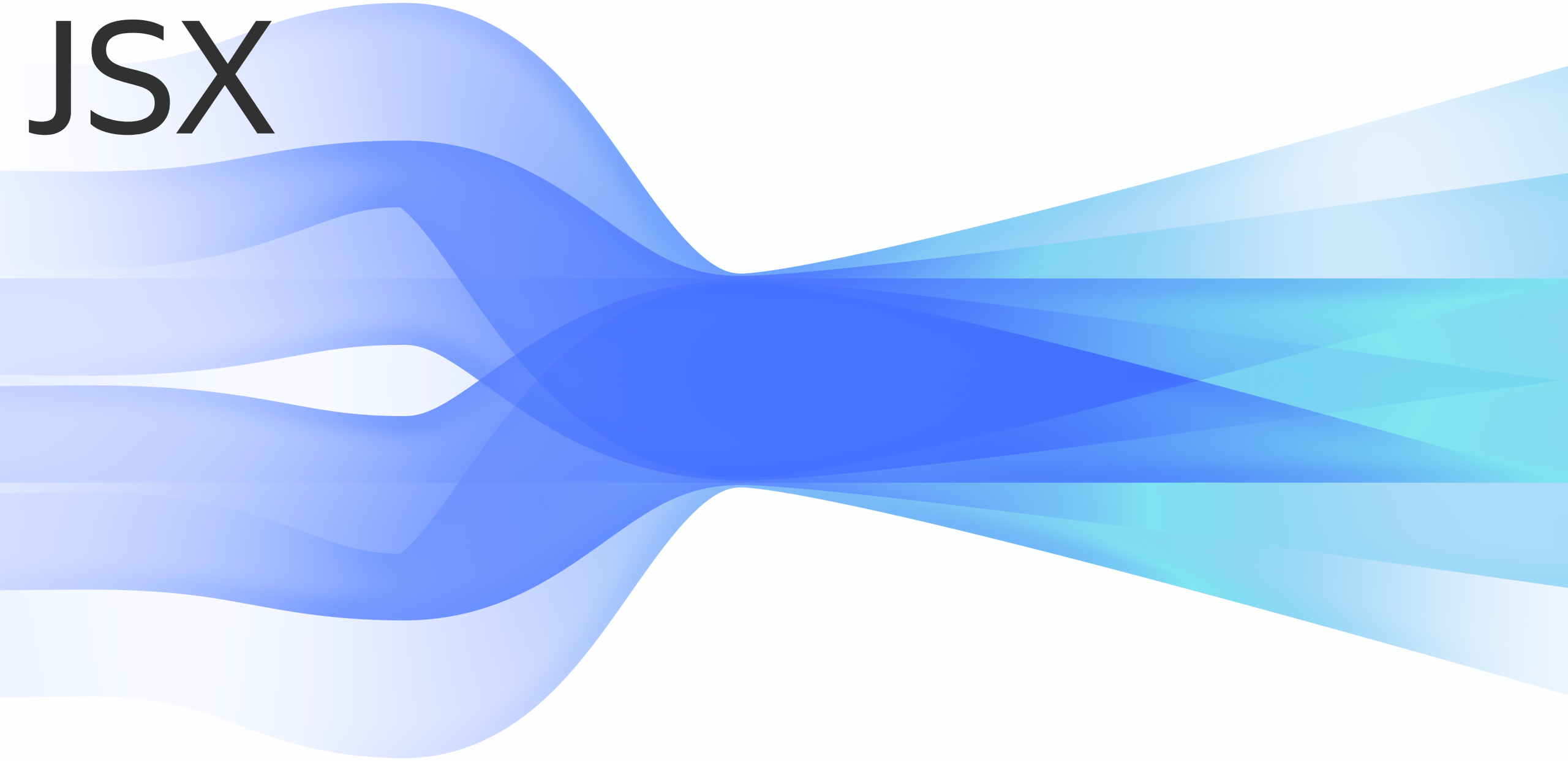
Шаг 2: добавление контекста

```
import { createContext } from 'react';

export const ThemeContext = createContext('light');

export const App = ({ children }) => {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme(() => theme === 'light' ? 'dark' : 'light');
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

Шаг 3: использование контекста

```
const Home = () => {
  const { toggleTheme, theme } = useContext(ThemeContext);

  return (
    <div>
      <label>Enable light theme</label>
      <input
        type="checkbox"
        onChange={toggleTheme}
        checked={theme === 'light'}
        className="inputBox"
      />
      <h1>Hello world!</h1>
    </div>
  );
};
```

JSX



Яндекс Образование

JSX

JSX (JavaScript XML) — синтаксическое расширение JavaScript, используемое в React для описания структуры интерфейса. Это гибрид HTML-подобного синтаксиса и JavaScript, позволяющий писать разметку прямо внутри кода компонентов.

Ключевое: JSX не является HTML и не выполняется браузером напрямую — это синтаксис, который на этапе компиляции преобразуется в вызовы React.createElement().

Особенности: JavaScript-выражения

Внутри JSX можно использовать JavaScript через фигурные скобки {}

```
const name = 'Аня';
const element = <p>Привет, {name}!</p>;


// Вычисления
const element = <p>Сумма: {2 + 3}</p>;

// Вызов функций
const element = <p>{formatDate(new Date())}</p>;
```

Особенности: атрибуты через camelCase

- class → className
- for → htmlFor
- tabindex → tabIndex

```
<input className="form-control" type="text" />
<label htmlFor="username">Имя</label>
```

Особенности: стили в объекте

CSS-свойства записываются в camelCase внутри объекта

```
const style = {
  backgroundColor: 'blue',
  color: 'white',
  padding: '10px'
};

<div style={style}>Текст со стилем</div>
```

Особенности: закрываем все теги

JSX строго требует закрывать теги (как XHTML)

```
<img src="logo.png" alt="Логотип" />
<br />
<input type="number" />
```

Особенности: возвращаем всегда один элемент

- Правильно

```
return (
  <div>
    <h1>Заголовок</h1>
    <p>Текст</p>
  </div>
);
```

- Неправильно

```
return (
  <h1>Заголовок</h1>
  <p>Текст</p>
);
```

Особенности: возвращаем всегда один элемент

- Можно даже так

```
return (
  <>
    <h1>Заголовок</h1>
    <p>Текст</p>
  </>
);
```

Пример: условия

```
function Greeting({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? (
        <p>Добро пожаловать!</p>
      ) : (
        <p>Пожалуйста, войдите.</p>
      )}
    </div>
  );
}
```

Пример: массив

```
function List({ items }) {
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}
```

Жизненный цикл

Яндекс Образование

Жизненный цикл

1. Mounting (монтирование) — компонент создаётся и впервые вставляется в DOM.
2. Updating (обновление) — компонент перерисовывается из-за изменения props или state.
3. Unmounting (демонтирование) — компонент удаляется из DOM.

Mounting (монтирование)

На этапе монтирования выполняются:

- Инициализация состояния через useState.
- Запуск побочных эффектов через useEffect с пустым массивом зависимостей.

Updating (обновление)

При изменении props/state происходит:

- Пересчёт JSX-разметки.
- Вызов useEffect — если зависимости в массиве изменились.
- Работа useMemo и useCallback для оптимизации.

Unmounting (демонтирование)

При удалении компонента из DOM:

- Выполняются cleanup-функции из useEffect и useLayoutEffect.

Работа с событиями

Яндекс Образование

Синтетические события

Синтетические события — кросс-браузерную абстракцию над native-событиями DOM. Они имитируют поведение стандартных событий, но имеют ряд важных особенностей

- Кросс-браузерность
- Асинхронный доступ: свойства события становятся null после завершения обработчика.

Привязка событий

События в React привязываются через пропсы с префиксом on

- Имя события — в camelCase (onClick, onMouseOver).
- Обработчик передаётся как функция (не строка).

```
<button onClick={handleClick}>Нажать</button>
<input onChange={handleChange} />
<form onSubmit={handleSubmit}>...</form>
```

Пример: получение данных из инпута

```
const handleChange = (event) => {
  console.log(event.target.value);
};

<input type="text" onChange={handleChange} />
```

Контролируемые компоненты

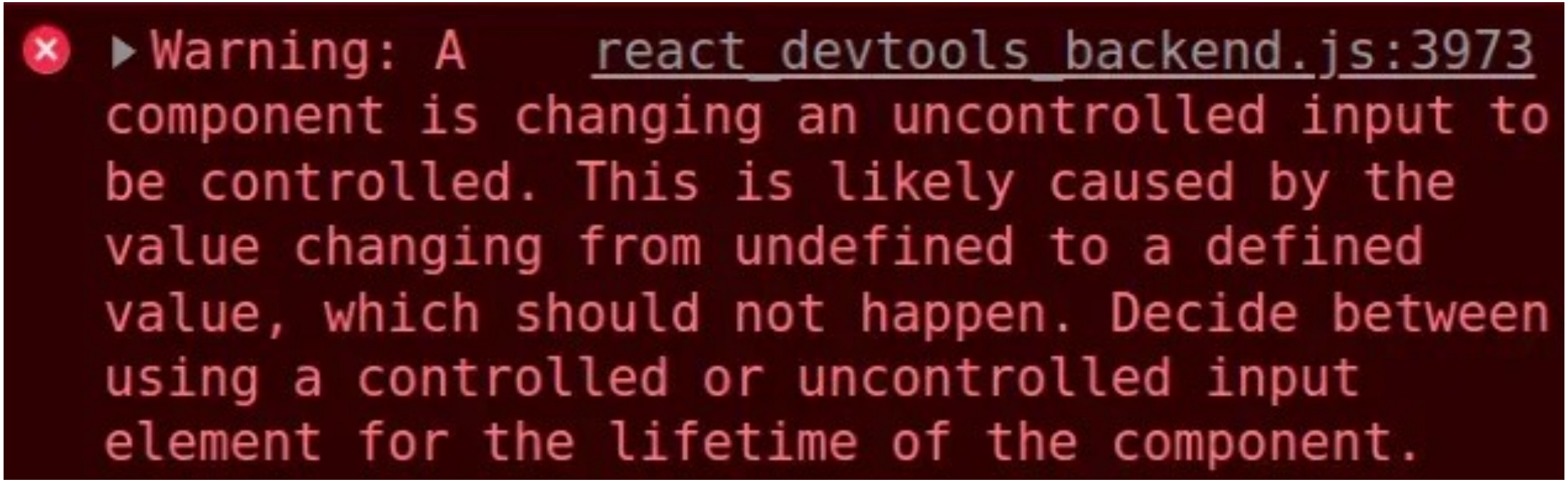
Контролируемый компонент — это элемент формы (input, textarea, select и др.), чьё значение полностью управляется состоянием React-компонента

1. Значение поля хранится в state компонента (через useState).
2. Любое изменение ввода обрабатывается через обработчик события (обычно onChange).
3. Обработчик обновляет состояние, а React перерисовывает поле с новым значением.
4. Состояние всегда синхронизировано с отображаемым значением поля.

Пример

```
function ControlledInput() {
  const [value, setValue] = useState('');

  const handleChange = (event) => {
    setValue(event.target.value);
  };

  return (
    <input
      type="text"
      value={value}           // Значение берётся из state
      onChange={handleChange} // Обработчик обновляет state
    />
  );
}
```

Ошибка


▶Warning: A component is changing an uncontrolled input to be controlled. This is likely caused by the value changing from undefined to a defined value, which should not happen. Decide between using a controlled or uncontrolled input element for the lifetime of the component.
react_devtools_backend.js:3973

Вопросы

Дополнительные ресурсы

- Официальный сайт https://react.dev/
- Курс от Кента Си Доддса https://www.epicreact.dev
- Ютуб-канал Михаила Непомнящего https://www.youtube.com/@mishanep
- Ютуб-канал Владилена Минина https://www.youtube.com/@VladilenMinin
- Метанит https://metanit.com/web/react/1.1.php